Ders 18 Çalışma Özeti

Ders 18 Çalışma Özeti

Genel Konular

- Rastgele sayılar konusu önceki derslerde bitmişti. Bu derste fonksiyonlar konusu örneklerle tekrar edilir:
  - Kesir işlemleri programının modüler tasarımı ve structure chart (yapı diyagramı).
  - Programın C'de kodlanması: `get_fraction`, `get_operator`, `SUM`, `MUL`, `refine`, `gcd`, `do_operation`.
  - Stub kavramı, Öklid'in OBEB algoritması, low coupling - high cohesion.
  - Akış diyagramında fonksiyon gösterimi.
  - Dizi ve matriste en küçük veya en büyük elemanı bulan genel fonksiyonlar.
  - Gelecek dersin konusu olan en yakın nokta çifti (closest pair) problemine giriş.
- Kesir işlemleri problemi
  - Kullanıcıdan iki kesir ($\frac{a}{b}$ ve $\frac{c}{d}$) ve bir operatör (`+`, `-`, `*`, `/`) alınır. Sonuç $\frac{x}{y}$ biçiminde, sadeleştirilmiş olarak yazılır. Örnek: $\frac{22}{38}$ değil, $\frac{11}{19}$.
  - Hoca öğrencilere sırayla soru sorarak programın hangi fonksiyonlara bölüneceğini tartışır.
- Tasarım tartışmasında çıkan fikirler
  - Toplama, çıkarma, çarpma ve bölme için birer fonksiyon, sadeleştirme için bir fonksiyon, ekrana yazdırma için bir fonksiyon.
  - Kesir alma işlemi iki kez tekrar edeceği için tek bir `get_fraction` fonksiyonu. Paydanın sıfırdan farklı olduğu bu fonksiyonda kontrol edilir.
  - Toplama ve çıkarma algoritmik olarak aynıdır, yalnızca işaret değişir. Ayrı bir işaret parametresi göndermeye de gerek yoktur: çıkarma için fonksiyon `SUM(a, b, -c, d, &x, &y)` olarak çağrılır.
  - Çarpma ve bölme de birleştirilebilir: $\frac{a}{b} \div \frac{c}{d} = \frac{a \cdot d}{b \cdot c}$. Bölme için çarpma fonksiyonu `c` ve `d` yer değiştirilerek çağrılır: `MUL(a, b, d, c, &x, &y)`. Önce $\frac{1}{c/d}$ hesaplamak gereksiz bir ek işlemdir.
  - Sonuçlar (`x`, `y`) iki tane olduğu için `return` ile döndürülemez. Adresleri gönderilir (call by reference), tıpkı `scanf`'te olduğu gibi.
  - Sadeleştirme için bir öğrenci sayının yarısından başlamayı önerdi. Hoca 11/22 örneğiyle bunun olmayacağını gösterip küçük sayıdan geriye doğru giderek ikisini birden bölen sayıyı aramayı söyledi. Bu aslında OBEB (GCD, greatest common divisor) algoritmasıdır.
  - Sadeleştirme fonksiyonu `x` ve `y`'nin adreslerini alır, sadeleşmiş değerler aynı değişkenlere yazılır.
  - `get_fraction` fonksiyonu `get_fraction(&a, &b)` şeklinde çağrılır. Fonksiyonun içinde parametreler zaten pointer olduğu için `scanf("%d", pay);` yazılır. `&pay` yazılırsa pointer'ın adresi, yani ikinci dereceden pointer gönderilmiş olur.
  - Ekrana yazdırma fonksiyonu değerleri değiştirmeyeceği için değişkenler adresle değil, değerle gönderilir.
- Structure chart (büyük olasılıkla Pearson ders kitabındaki Şekil 6.14)
  - En üstte ana modül (main veya bir test/driver fonksiyonu) vardır. Altında `Get fraction problem`, `Continue?`, `Compute result` ve `Print problem and result` modülleri bulunur.
  - `Get fraction problem` altında `scan_fraction` (pay ve paydayı döndürür) ve `get_operator`. `Compute result` altında `add_fractions`, `multiply_fractions` ve `re-`

duce_fraction. reduce_fraction de find_gcd'yi çağırır. Print problem and result altında print_fraction vardır.
  - Oklar her fonksiyona giren parametreleri ve fonksiyondan dönen değerleri gösterir. Yazdırma modülüne yalnızca giriş vardır, dönüş yoktur.
  - Structure chart programdaki fonksiyonları ve parametrelerini önceden gösterir. Hangi fonksiyonların birleştirilebileceğini kağıt üzerinde görmeyi sağlar. Hoca projelerde bu tür bir diyagram çizilmesini önerir.
- Programın kodlanması (Commonfractions.c)
  - void get_fraction(int *pay, int *payda): pay scanf("%d", pay) ile alınır. Payda do { scanf("%d", payda); } while (*payda == 0); ile sıfır olmayana kadar tekrar istenir. Koşulda payda değil *payda yazılmalıdır, çünkü adresin değil içeriğin kontrolü yapılır.
  - char get_operator(): tek bir değer döndürdüğü için void yerine char döndürür, pointer'a gerek kalmaz. Okuma do { scanf(" %c", &c); } while (c != '+' && c != '-' && c != '*' && c != '/'); ile yapılır. Dört operatörden biri girilene kadar tekrar sorulur.
  - %c'den önceki boşluk, önceki girişten bufferda kalan Enter'ın okunmasını engeller. Sonraki bir çalıştırmada boşluk unutulduğu için "Operatör giriniz" mesajı iki kez göründü.
  - print_fraction(int x, int y): printf("%d/%d", x, y) ile kesri yazar. Ana fonksiyon önce birinci kesri, sonra operatörü, sonra ikinci kesri yazdırır.
  - Pointer kullanan fonksiyonlar için prototip yazılmalıdır. Prototipte değişken adları silinebilir ama tipler (int *, dizi için [], matris için [][SUTUN]) kalmalıdır.
  - void SUM(int a, int b, int c, int d, int *x, int *y): *x = a*d + b*c; *y = b*d; ve ardından refine(x, y);.
  - void MUL(...): *x = a*c; *y = b*d; ve refine(x, y);.
  - + gibi bir sembol fonksiyon adı olamaz. Değişken adlarında olduğu gibi kurallara uyan bir ad (ör. SUM) seçilir.
  - void refine(int *x, int *y): g = gcd(*x, *y); *x = *x / g; *y = *y / g;.
  - int gcd(int x, int y): küçük olan sayı min = x < y ? x : y; ile bulunur (koşul operatörü). while (x % min != 0 || y % min != 0) min--; ile ikisini birden bölen ilk sayıya kadar geriye gidilir. En kötü durumda min 1 olur ve döngü biter. Küçük sayının yarısından başlamak yanlış olur: $\frac{11}{22}$'de OBEB 11'in kendisidir.
  - do_operation(a, b, c, d, op, &x, &y): if-else if ile operatöre göre SUM veya MUL çağrılır. Daha sonra refine çağrısı SUM ve MUL'den çıkarılıp yalnızca do_operation'ın sonuna konuldu. Böylece tekrar eden kod (duplicated code) kalktı ve refine'ın kendisinde hiçbir değişiklik gerekmedi.
  - Fonksiyon pointer'ları ikinci sınıfta görülecektir. Bu tür seçimleri daha kısa yazmayı sağlarlar.
  - Denemeler: 2/3 + 2/3 = 4/3 (12/9 sadeleşti). 2/3 ÷ 2/3 = 1/1. Çıkarmada 2/3 - 2/3 sonucunda program hata verdi. Hoca bunun OBEB ile ilgili ayrı bir sorun olduğunu ve düzeltilmesi gerektiğini söyler.
  - Negatif sayılar sorulduğunda hoca, OBEB hesaplanmadan önce sayıların mutlak değerlerinin alınması gerektiğini söyler.
- Kitaptaki çözüm
  - main'de sonuç hesaplanıp reduce_fraction çağrılır, problem ve sonuç yazdırılır, kullanıcıya devam etmek isteyip istemediği sorulur.
  - Kitap, pay ile paydanın çarpımı negatifse işaretleri düzenleyerek negatif durumları ele alır. Hoca "çarpımları sıfırdan farklıysa, yani bir tanesi negatifse" dedi; koşul büyük olasılıkla sıfırdan küçük olmasıdır.
  - Kitabın get_operator fonksiyonu Enter karakterini ayrıca işlediği için gereksiz ayrıntılıdır. Yalnızca koşul içeren bir for (; koşul ;) döngüsü kullanılmış. for'da başlangıç ve artırma kısımları boş bırakılabilir.
  - find_gcd bir stub'dır: OBEB'i hesaplamaz, sayıları ekrana yazıp OBEB'i kullanıcıdan ister. Ekip çalışmasında başka birinin yazacağı bir fonksiyon henüz hazır değilken kendi modülünüzü test etmek için geçici olarak böyle bir fonksiyon yazılır.
- Fonksiyonları ayırmanın gerekçesi
  - refine ile gcd neden ayrı yazıldı? OBEB başka yerlerde de kullanılabilir ve algoritması değiştirilmek istenebilir. gcd refine'ın içine yazılırsa bağımlılık (coupling) artar. Her fonksiyon adına uygun tek bir iş yapmalıdır (low coupling, high cohesion).

  - Öklid algoritması: gcd$(18, 10)$: $18 \bmod 10 = 8$, $10 \bmod 8 = 2$, $8 \bmod 2 = 0$, sonuç 2. Geriye doğru deneme yöntemi 10'dan 2'ye kadar gider. Öklid birkaç adımda bulur. Algoritmayı iyileştirmek için yalnızca `gcd` fonksiyonunu değiştirmek yeter.
  - Bir tasarım değişikliği bütün programın baştan yazılmasını gerektiriyorsa tasarım kötüdür.
- Akış diyagramında fonksiyon
  - Fonksiyonun başlangıcına sola bakan bir ok çizilir. Okun üstüne fonksiyon adı ve giriş parametreleri (ör. `MaxBul(A, N)`), altına çıkış değerleri (ör. `maxD, maxIndis`) yazılır. Fonksiyon `stop` ile biter.
  - Akış diyagramında, MATLAB'da ve Python'da birden fazla değer döndürülebilir. C, C++, Java ve C#'ta `return` tek değer döndürür. Birden fazla değer gerekiyorsa call by reference kullanılır.
  - Fonksiyonlar parametrik yazılmalıdır: 4 elemanlı değil n elemanlı diziyi okuyan fonksiyon. Artan veya azalan sıralama gibi seçenekler bir flag parametresiyle tek fonksiyonda toplanabilir.
- Dizi ve matriste en küçük veya en büyük eleman (`soru2.c`)
  - `int DiziMinMaxBul(int dizi[], int n, int op, int *indis)`: `op == 1` ise en büyüğü, değilse en küçüğü bulur. Değeri `return` ile, indisi pointer ile döndürür. Aslında indis bilinirse değere de ulaşılır, ama soru `return` kullanılmasını istiyor diye kabul edildi. Derste 9 değeri ve 4 indisi bulundu.
  - `if` dışarıda, iki `for` içeride yazıldı. Böylece döngünün her turunda `op` kontrol edilmez.
  - `int MatMinMaxBul(int mat[][50], int m, int n, int op)`: her satır `tmp` dizisine kopyalanır ve `DiziMinMaxBul` ile o satırın en büyüğü veya en küçüğü `K[i]`'ye yazılır. Sonra `K` dizisi için aynı fonksiyon bir kez daha çağrılır. `int K[50] = {0};` ile dizi sıfırlanır.
  - Bu, var olan bir çözümü başka bir probleme uyarlamaktır: transform and conquer. Matris önce diziye indirgenir.
  - 3 × 4'lük matris (2 3 4 5 / 6 7 8 1 / 3 4 5 9) için en büyük eleman 9 bulundu.
- En yakın nokta çifti (closest pair)
  - Düzlemdeki (veya n boyutlu uzaydaki) noktalardan birbirine en yakın ikisini bulmak. Uzaklık temelli yapay zeka sistemleri ve öneri sistemleri bu esasa dayanır. Hoca doktora tezinde de kullanmış.
  - Örnek: her öğrenci matematik ve tarih puanlarına göre düzlemde bir noktadır. Birbirine en yakın iki nokta, başarı profili en çok benzeyen iki öğrencidir.
  - Kaba kuvvet çözümü: `min = INT_MAX`, `i` ve `j` 1'den 1000'e, `d(i, j) < min` ise `min = d(i, j)` ve indisleri sakla. n^2 çalışır.
  - Boyut sayısı arttıkça (20 ders) uzaklık hesabına yeni boyutlar eklenir.
  - $n \log n$'de çalışan algoritmalar vardır. Bunlar Algoritma Analizi dersinin konusudur. Gelecek ders n^2 çözüm üzerinde iyileştirmelere bakılacak.
- Duyurular
  - Gelecek ders yaklaşık iki hafta sonradır (bayram tatili). Bu derste closest pair ve fonksiyonlarla ilgili algoritmik örnekler işlenecektir. Cumartesi katılmak isteyenlerle ayrıca görüşülecek.
  - Ders kayıtlarının linki diğer gruptan bir öğrenci tarafından Telegram'da paylaşılmış. Slaytların PDF'i yüklenecek.

Hocanın Özellikle Vurguladığı Kısımlar

- Programı kodlamadan önce fonksiyonlarını ve parametrelerini belirleyin. Structure chart bunu sistematik hale getirir.
- Algoritmik olarak aynı olan işlemleri (toplama/çıkarma, çarpma/bölme) tek fonksiyonda birleştirin. Farkı çağırırken verdiğiniz parametrelerle sağlayın.
- Fonksiyon içinde değeri değişmeyecek parametreleri adresle göndermeyin.
- Fonksiyon parametresi zaten pointer ise `scanf`'te önüne `&` koymayın. Koşullarda içeriğe `*` ile bakın.
- Her fonksiyon tek bir iş yapmalı. OBEB'i sadeleştirmeden ayırın ki değiştirilebilsin ve başka yerlerde kullanılabilsin.
- Tekrar eden kodu tek bir yere taşıyın. İyi bir tasarımda bir değişiklik programın geri kalanını bozmaz.

Kısa Tekrar Notları

- $\frac{a}{b} + \frac{c}{d} = \frac{ad+bc}{bd}$. Çıkarma: `SUM(a, b, -c, d, &x, &y)`.
- $\frac{a}{b} \cdot \frac{c}{d} = \frac{ac}{bd}$. Bölme: `MUL(a, b, d, c, &x, &y)`.
- Sadeleştirme: `g = gcd(x, y)`, `x /= g`, `y /= g`.
- Öklid: `while (y != 0) { r = x % y; x = y; y = r; }`, sonuç `x`.
- `scanf(" %c", &c)`: boşluk, bufferdaki Enter'ı atlar.
- Operatör kontrolü: `while (c != '+' && c != '-' && c != '*' && c != '/')`.
- Stub: başka bir modülü test etmek için yazılan geçici, sahte fonksiyon.
- Matrisin en büyüğü: her satırın en büyüğünü bir diziye koy, o dizinin en büyüğünü bul.

Detaylı Açıklamalar

Kesir programının çekirdeği

Derste yazılan fonksiyonlar ders sonrasında bir test programında toplandı ve `gcc -Wall -std=c89 -pedantic` ile denendi. `refine`'daki pay 0 kontrolü ve mutlak değer derste yoktu, aşağıdaki notlara göre eklendi:

```c
void do_operation(int a, int b, int c, int d, char op, int *x, int *y){
    if (op == '+') SUM(a, b, c, d, x, y);
    else if (op == '-') SUM(a, b, -c, d, x, y);
    else if (op == '*') MUL(a, b, c, d, x, y);
    else MUL(a, b, d, c, x, y);
    refine(x, y);
}
void SUM(int a, int b, int c, int d, int *x, int *y){ *x = a*d + b*c; *y = b*d; }
void MUL(int a, int b, int c, int d, int *x, int *y){ *x = a*c; *y = b*d; }
void refine(int *x, int *y){
    int g;
    if (*x == 0) { *y = 1; return; }   /* derste yazilmadi */
    g = gcd(*x < 0 ? -*x : *x, *y < 0 ? -*y : *y);
    *x = *x / g; *y = *y / g;
}
int gcd(int x, int y){
    int min = x < y ? x : y;
    while (x % min != 0 || y % min != 0) min--;
    return min;
}
```

Çıktılar: 2/3 + 2/3 = 4/3, 2/3 - 2/3 = 0/1, 2/3 * 2/3 = 4/9, 2/3 / 2/3 = 1/1, 3/4 + 4/6 = 17/12.

`DiziMinMaxBul` ve `MatMinMaxBul` aynı programda denendi: 3 7 1 5 9 için en büyük 9 (indis 4), en küçük 1 (indis 2). Derste girilen matris için en büyük 9, en küçük 1.

Ders sonrası notlar

- **2/3 - 2/3 neden çöktü?** Sonucun payı 0 olur. `gcd(0, 9)`'da `min` 0 olur ve `x % 0` hesaplanır. Sıfıra bölme programı çökertir. Ders sonrasındaki denemede derste yazılan `gcd(0, 9)` `Floating point exception` ile kapandı. Çözüm: pay 0 ise OBEB aramadan sonucu 0/1 yapmak. Öklid algoritması `gcd(0, y) = y` verdiği için bu durumu kendiliğinden doğru ele alır.
- **Sıfır kesirle bölme:** `get_fraction` yalnızca paydaların 0 olmasını engeller. $\frac{a}{b} \div \frac{0}{d}$ işleminde `MUL(a, b, d, c, ...)` çağrısı sonucun paydasını $b \cdot 0 = 0$ yapar. Bölme seçildiğinde ikinci kesrin payının da 0'dan farklı olduğu kontrol edilmelidir.
- **Negatif sayılar ve OBEB:** Hoca, negatif sayılarda mutlak değer alınması gerektiğini söyledi. Bu doğrudur, aksi halde `min` negatif olur. Ayrıca sonucun işareti paydada kalmamalıdır: payda negatifse pay ve paydanın ikisinin de işareti değiştirilir. Kitabın çözümü bunu yapar.

- **12/9'un OBEB'i:** Derste bir an "OBEB 9" dendi. 12 ile 9'un OBEB'i 3'tür. 12/9 = 4/3 sonucu da bunu doğrular.
- **`if (x == y)` gereksiz:** Hoca `gcd`'ye önce eşitlik kontrolüyle başladı (01:11:32), sonra koşul operatörüne geçti. `x < y ? x : y` ifadesi eşitlikte de doğru sonucu (`y`, yani aynı sayıyı) verir, bu yüzden ayrı bir `if`'e gerek yoktur.
- **Operatör kontrolünde `&&`:** Derste bir ara koşulun "veya" ile yazılıp yazılmayacağı tartışıldı. "Dört operatörden hiçbiri değilse tekrar sor" demek, De Morgan kuralına göre `c != '+' && c != '-' && c != '*' && c != '/'` demektir. `||` ile yazılırsa koşul her zaman doğru olur ve döngüden hiç çıkılamaz.
- **Kaba kuvvet closest pair:** Tahtadaki taslakta `i` ve `j` 1'den 1000'e kadar gidiyordu. Bu durumda `i == j` için uzaklık 0 olur ve en küçük uzaklık her zaman 0 çıkar. `j` `i + 1`'den başlatılmalıdır. Bu hem hatayı giderir hem de her çifti bir kez inceler: $\frac{n(n-1)}{2}$ karşılaştırma. Ayrıca her nokta en az bir kez okunmak zorunda olduğu için bu problem $\log n$ veya sabit zamanda çözülemez. En iyi bilinen genel çözüm, böl ve yönet ile çalışan $n \log n$ algoritmasıdır.
- **Dönüş değeri:** Derste programın sonunda büyük olasılıkla "Process exited … with return value 2" görüldü. `main`'in sonunda `return 0;` olmadığında C89'da dönüş değeri belirsizdir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.